

# Build Steps

Part 1 of 5 — Set up the agent, orchestration, and knowledge

## 1 Create Your Copilot Studio Agent

Open Copilot Studio ([copilotstudio.microsoft.com](https://copilotstudio.microsoft.com)), select Create → New agent, then configure:

|                               |                                                                                                                                                                  |
|-------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Agent Name</b>             | Incident Postmortem Copilot                                                                                                                                      |
| <b>Description</b>            | Drafts engineering incident postmortems from incident logs, identifies recurring root-cause patterns, suggests action items, and never assigns individual blame. |
| <b>Solution / Environment</b> | Engineering Operations · Default environment                                                                                                                     |
| <b>Icon</b>                   | Use a recognisable colour and emoji or upload a brand icon                                                                                                       |

## 2 Choose Orchestration Mode

Under Settings → Generative AI, choose how the agent decides what to do:

### Generative Orchestration

*Recommended — AI-led*

- Agent reasons over user input and picks the right topic, tool, or knowledge
- Handles paraphrases, multi-step asks, and follow-ups naturally
- Best for varied user intent and exploratory questions
- Requires clear instructions, tool descriptions, and good knowledge

### Classic Orchestration

*Deterministic — trigger-led*

- Topics fire on explicit trigger phrases you author
- Predictable flows, easier to audit step-by-step
- Best for tightly-scoped, regulated, or compliance-critical journeys
- Less forgiving of phrasing the author didn't anticipate

*For this build we use Generative Orchestration so the agent can blend knowledge lookups with tool calls for incident response and blameless postmortems.*

## 3 Add Knowledge

Go to the Knowledge tab → Add knowledge → File, then upload Tech\_Incidents\_Data.xlsx. This file contains:

- Incident metadata (incident ID, severity SEV1-4, service, environment Prod/Stage)
- Timeline (start UTC, detected UTC, resolved UTC, MTTR minutes)
- Root cause (root cause category, summary)
- Follow-up (action items open, owning team)

*You can layer additional sources later — SharePoint sites, Dataverse, public web, or Graph connectors.*

# Build Steps

Part 2 of 5 — Add instructions and a custom topic

## 4 Add Agent Instructions

Open Overview → Instructions and paste the following (these guide the model regardless of orchestration mode):

You are the Incident Postmortem Copilot. You help two audiences:  
1) Engineers writing a postmortem after an incident.  
2) Engineering leaders reviewing root-cause patterns across the estate.

Rules:

- Use the uploaded incidents dataset as your single source of truth.
- Blameless language always – describe systems, signals and decisions, never individuals. Replace any name in source notes with 'on-call engineer' or 'owning team' before quoting back.
- Quote incident IDs and timestamps verbatim – no rounding.
- For postmortem filing, call the tool – never just claim it was filed.
- Highlight recurring root cause categories (3+ in 60 days).

## 5 Add a Custom Topic

Go to Topics → Add a topic → Create from description. Topics are reusable conversation flows the agent can call when intent matches.

Topic name: Draft Incident Postmortem

Description: Walks an engineer through drafting a blameless postmortem for a chosen incident, then files it as a work item in Azure DevOps for the owning team.

Trigger phrases (Generative Orchestration uses these as intent hints, Classic mode uses them as exact triggers):

- Draft a postmortem for INC-2451
- Help me write up the SEV2 from yesterday
- Postmortem for the checkout outage
- Start an incident retro

Conversation steps:

- Ask for the IncidentID – confirm one row matches in knowledge
- Render structured draft: Summary · Timeline · Impact · Root cause · Action items
- Pull from row: StartUTC, DetectedUTC, ResolvedUTC, MTTRMinutes, RootCauseCategory
- Suggest 3-5 action items in blameless language, owned by team not person
- Read it back; ask the engineer to confirm or edit any section
- Call 'File Incident Postmortem in Azure DevOps' with the final draft

# Build Steps

Part 3 of 5 — Add a tool and configure settings

## 6 Add a Tool

Tools let the agent take an action, not just answer. Go to Tools → Add a tool, and choose one of:

- Prebuilt connector (Outlook, Teams, SharePoint, Dataverse, ServiceNow, etc.)
- Power Automate cloud flow (custom logic you author or already own)
- Custom HTTP request (any REST API your agent can reach)
- Model Context Protocol (MCP) server — for richer tool ecosystems

Tool name: File Incident Postmortem in Azure DevOps  
Type: Azure DevOps connector (Create work item – type 'Incident Postmortem')  
Description: Creates a postmortem work item on the owning team's board with the drafted summary, action items as child tasks, and links back to the incident ID.

Inputs the agent will collect or infer:

- Incident ID (string)
- Severity (SEV1 / SEV2 / SEV3 / SEV4)
- Owning team (string, from knowledge)
- Postmortem markdown body (string – sections Summary / Timeline / Impact / Root Cause)
- Action items list (string array – each becomes a child work item)

Outcome: Returns the Azure DevOps work item URL and posts a card to the owning team's Teams channel for visibility.

## 7 Configure Settings

Open the Settings cog (top right). Configure these four areas before you let real users near the agent:

- General — confirm name, description, and primary language
- Generative AI — orchestration mode and content moderation level (start at Medium)
- Authentication — use 'Authenticate with Microsoft' so the agent runs as the signed-in user
- Security — review DLP policy, enabled channels, and end-user authentication

# Build Steps

Part 4 of 5 — Evaluate, test, and publish to a channel

## 8 Evaluate Your Agent

Go to Analytics → Evaluations → New evaluation. Build a test set so you can measure quality before and after every change.

Test set name: Incident Postmortem Copilot – Baseline v1  
Approach: Pair each input prompt with an expected outcome.  
Run the evaluation and review three scores per response:

- Groundedness – did the answer stay inside the knowledge?
- Relevance – did it actually address the question?
- Accuracy – did it match the expected outcome?

Sample evaluation items:

Input: Draft a postmortem for INC-2451

Expect: Returns blameless structured draft, no names; timeline matches row verbatim.

Input: What root cause categories repeat in the last 60 days?

Expect: Counts RootCauseCategory occurrences within 60 days, flags any with count >= 3.

Input: File the postmortem for INC-2451 to Azure DevOps for the Checkout team

Expect: Triggers Draft Incident Postmortem topic and calls the Azure DevOps tool.

## 9 Test in the Test Pane

Use the Test pane (right side of the canvas) while you build. For each query, click Activity map to inspect what the agent did:

- Which topic, tool, or knowledge source fired (and in what order)
- Inputs the agent collected and outputs the tool returned
- Citations linking each answer back to a row of your knowledge file
- Latency per step — useful when tuning multi-tool flows

## 10 Publish to a Channel

When you're happy, click Publish (top right). Then open Channels and enable the surface your users live in. Recommended for this build: Microsoft Teams (Engineering channels) — link postmortems back to team boards.

- Microsoft Teams — pin to each engineering team channel
- Microsoft 365 Copilot — for leaders reviewing root cause patterns
- Custom website — embed on the Engineering Excellence portal
- MCP server — expose the agent's tools to your existing dev tooling stack

*Teams and M365 Copilot channels require tenant admin approval before users see the agent — plan for that lead time.*

# Try These Live

Part 5 of 5 — Sample queries to test your agent during the build-along.

## Postmortem Drafting

Draft a postmortem for INC-2451.  
Give me a blameless summary of yesterday's SEV2.

## Pattern Detection

What root cause categories repeat in the last 60 days?  
Which service has the worst MTTR in production this quarter?

## Governed Action

File the postmortem for INC-2451 as a work item on the Checkout team's board.  
Try a paraphrase: 'ship the retro for the payment timeout incident'.

## How to Think About This Agent

### This agent replaces:

- Engineers writing postmortems from a blarney
- Manual root-cause tagging in spreadsheets
- Action items lost in chat threads

### With:

- ✓ Consistent, blameless postmortems on every incident
- ✓ Action items tracked as work items, not memory
- ✓ Recurring root causes surfaced for leadership

**Trusted, governed, enterprise AI — built once, used everywhere.**

### Tip for real-world use

Add an MCP server alongside the Azure DevOps tool so the agent can also pull live metrics from your observability stack (e.g. error rates, latency p95) into the Impact section automatically. Run the evaluation set after every model upgrade.

## You've built a Copilot Studio agent ✓

You now have an AI agent that can:

- Draft blameless postmortems on demand
- Detect recurring root causes
- File work items for every action
- Tighten the incident learning loop